

LLM Notes

2026 Spring

Day 17: PyTorch GPU Memory Snapshot

CS336 Systems

主讲: Aaron

Fudan University

今日摘要

2026.7.19 PyTorch Memory Snapshot 用于记录 CUDA allocator 管理的显存块及其生命周期。通过 Active Memory Timeline, 可以观察模型参数、forward activation、backward gradient 和临时 tensor 如何共同形成显存峰值。

实验表明, 短序列下参数和梯度占据主要显存; 长序列下, attention 中随 S^2 增长的临时 tensor 会形成明显的周期性峰值, 并可能直接导致 OOM。

17.1 Memory Snapshot 是什么

PyTorch 可以记录 CUDA allocator 的显存分配历史, 并将结果保存为 `.pickle`:

```
torch.cuda.memory._record_memory_history()
```

```
# Run the operation being measured.
```

```
torch.cuda.memory._dump_snapshot(snapshot_path)
```

```
torch.cuda.memory._record_memory_history(enabled=None)
```

`pickle` 是 Python 的序列化文件格式。这里保存的不是图片, 而是显存 allocation、deallocation、大小、地址和调用栈等结构化信息。该文件可以上传到 PyTorch Memory Visualizer, 进行可视化。

17.2 如何阅读 Active Memory Timeline

Active Memory Timeline 中:

- 横轴表示 allocation 和 deallocation 事件发生的顺序或时间;

- 纵轴表示当前仍处于 active 状态的显存总量；
- 每条彩色带表示一个显存 allocation；
- 彩色带的高度对应该 allocation 的大小；
- 彩色带的宽度对应该 allocation 保持存活的时间；
- 总轮廓的最高点表示 active memory peak。

17.2.1 Entries 的含义

界面中的 **entries** 表示当前 snapshot 中可供显示的显存记录条目。这些条目通常对应 allocator 记录的显存块或 allocation event。

17.3 实验设置

我们对 XL 模型进行了以下组合的显存实验：

Sequence length	Precision	Mode
128	FP32	Forward / Train Step
128	BF16 autocast	Forward / Train Step
2048	FP32	Forward / Train Step
2048	BF16 autocast	Forward / Train Step

每次 memory profiling 只记录一个 measurement step：

只记录一次迭代可以使时间线更清晰，并避免多个重复 iteration 混在一起。

17.4 短序列： $S = 128$

在 $S = 128$ 的 forward-only 实验中，整体曲线接近平坦。原因是：

- XL 模型参数形成了较大的长期显存基线；
- $S = 128$ 时 activation 相对较小；
- 临时 tensor 生命周期很短，并不断被 allocator 重用；
- 在完整纵轴尺度下，这些短暂变化不容易被看见。

这不表示 forward 没有产生临时 tensor，而是这些临时分配相对于模型参数基线较小。

17.4.1 FP32 Train Step

FP32 train step 中观察到的关键显存节点约为：

阶段	Active memory
记录开始	50.5 GiB
清除上一轮梯度后	38.1 GiB
Forward 保存 activation 后	43.6 GiB
Backward 附近峰值	50.9 GiB

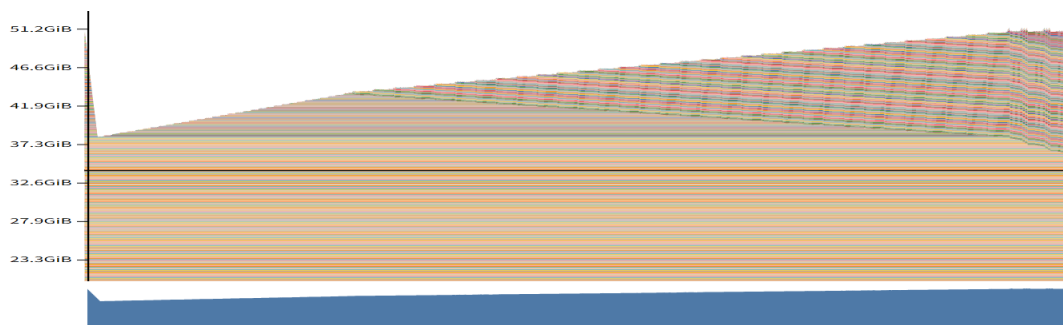


图 1: XL 模型在 FP32、 $S = 128$ 下的 train-step active memory timeline。显存先因清除梯度下降，随后在 forward 和 backward 中逐渐上升。

该过程可以理解为：

1. warmup 后仍存在参数、梯度和其他长期 allocation；
2. `zero_grad(set_to_none=True)` 释放上一轮梯度；
3. forward 为未来的 backward 保存 activation；
4. backward 逐层释放 activation，同时创建参数梯度；
5. optimizer update 进一步访问或初始化 optimizer states。

因此，训练显存并不是简单地把参数、梯度和 activation 静态相加；不同 tensor 的生命周期会发生重叠和替换。

17.5 长序列： $S = 2048$

当 sequence length 从 128 增长到 2048 后，forward 曲线出现了大量周期性尖峰。

每个 Transformer block 都会执行一次 attention 和 MLP，因此不同层会重复产生形状相似的临时 tensor。模型包含多个 block，于是时间线上出现周期性的显存峰值。

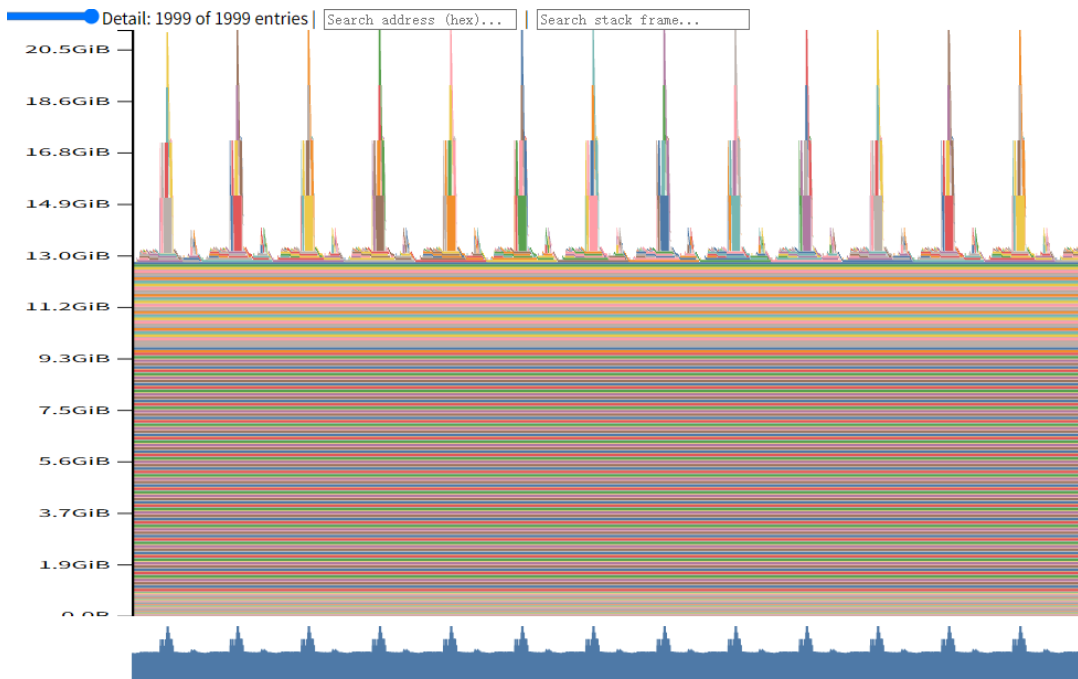


图 2: XL 模型在 FP32、 $S = 2048$ 下的 forward active memory timeline。周期性尖峰对应不同 Transformer block 中的临时 activation。

17.5.1 约 320 MiB 的 MLP Tensor

XL 配置中:

$$B = 4, \quad S = 2048, \quad d_{\text{ff}} = 10240.$$

一个 FP32 MLP 中间 activation 的大小约为:

$$\begin{aligned} M_{\text{MLP}} &= BSd_{\text{ff}} \times 4 \text{ bytes} \\ &= 4 \times 2048 \times 10240 \times 4 \text{ bytes} \\ &= 320 \text{ MiB}. \end{aligned}$$

这与 snapshot 中观察到的较小尖峰基本一致。

17.5.2 约 2 GiB 的 Attention Tensor

对于普通 multi-head attention, attention score 的形状为:

$$(B, H, S, S).$$

在当前配置中：

$$B = 4, \quad H = 32, \quad S = 2048.$$

如果 attention score 使用 FP32，则大小为：

$$\begin{aligned} M_{\text{attention}} &= BHS^2 \times 4 \text{ bytes} \\ &= 4 \times 32 \times 2048^2 \times 4 \text{ bytes} \\ &= 2 \text{ GiB}. \end{aligned}$$

这与 snapshot 中观察到的大尖峰一致。

分析

MLP activation 的大小随 S 线性增长：

$$M_{\text{MLP}} = O(BSd_{\text{ff}}),$$

而普通 attention score 随 S^2 增长：

$$M_{\text{attention}} = O(BHS^2).$$

因此，sequence length 较大时，attention tensor 会快速成为显存瓶颈。

17.6 FP32 与 BF16 的比较

在 $S = 2048$ 的 forward 实验中，BF16 的峰值低于 FP32：

原因是 autocast 让部分矩阵乘法输出和中间 activation 使用 BF16，每个元素由 4 bytes 降为 2 bytes。

但是显存不会整体减半，因为：

- 模型参数仍然是 FP32；
- 部分数值敏感运算仍然使用 FP32；
- 自定义 softmax 可能产生 FP32 attention 临时 tensor；
- CUDA allocator、workspace 和其他固定开销不会按精度减半。

这也解释了为什么 $S = 128$ 时两种精度的图像相近：短序列 activation 较小，显存主要被不随 autocast 改变的 FP32 参数及其他固定部分占据。

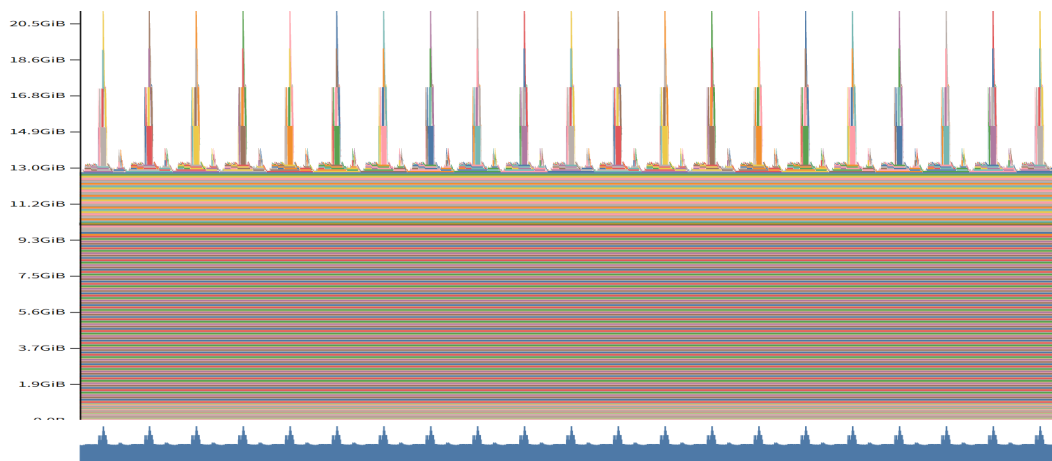


图 3: XL 模型在 BF16 autocast、 $S = 2048$ 下的 forward active memory timeline。部分 activation 使用 BF16，因此峰值低于 FP32。

17.7 OOM 结果

以下两个实验在单张约 80 GiB GPU 上发生 OOM:

- XL, $S = 2048$, FP32, train step;
- XL, $S = 2048$, BF16 autocast, train step。

OOM 发生在 attention softmax 附近，程序尝试继续分配约 2 GiB tensor。